CCAR-F Cheat Sheet

AGENTIC ARCHITECTURE

27% OF EXAM

Agents | Routing | Chaining | Parallelism | Subagents

CLAUDE CODE

20% OF EXAM

Memory | Settings | Permissions | Hooks | CI

PROMPTS + OUTPUT

20% OF EXAM

System | Examples | Schemas | JSON | Stops

TOOLS + MCP

18% OF EXAM

Schemas | Tools | Resources | Prompts | Transport

CONTEXT + RELIABILITY

15% OF EXAM

RAG | Caching | Evals | Retries | Guardrails

QUICK FACTS

CREDENTIAL CCAR-F	ISSUER Anthropic
ACCESS Partner network	FOCUS Production architecture
USERS Claude architects	CORE Claude ecosystem
VERIFY Exam portal	

SIMPLE FIRST

AG

Call, retrieve, chain, route, agent.

Call | Retrieve | Chain | Route | Agent

WORKFLOW VS AGENT

AG

Workflow	Agent
Predictable sequence	Dynamic path
Easier control	Higher autonomy

Prefer simplest control

PATTERN PICKER

AG

Task is simple	→	Single call
Needs facts	→	RAG call
Steps known	→	Prompt chain
Input classes known	→	Router
Subtasks independent	→	Parallel calls
Steps unknown	→	Agent loop
Quality measurable	→	Evaluator optimizer
Risk high	→	Human review

ARCHITECTURE PATTERNS

AG

Single call
Best baseline

RAG call
Retrieve then answer

Prompt chain
Ordered model steps

Router
Classify then dispatch

Parallel
Independent subtasks

Orchestrator
Plan then delegate

Evaluator
Judge then improve

AGENT CONTROLS

AG

Loop limit
Bound tool cycles

Role scope
Narrow responsibility

Tool scope
Least privilege

State
Track decisions

Handoff
Define output contract

Escalation
Ask human

Trace
Log key steps

CLAUDE CODE

CO

CLAUDE.md
Project memory

settings.json
User or shared project settings

Permissions
Gate risky actions

Slash command
Reusable workflow

Hook
Run workflow action

Subagent
Specialized helper

Headless
Noninteractive run

SCHEMA VS PROSE

PR

Schema	Prose
Machine parsed	Human read
Validated fields	Flexible wording

Parsing needs schema

PROMPT BASICS

PR

System
Durable constraints

Task
Goal and inputs

Criteria
Success definition

Examples
Show desired shape

Context
Relevant facts

Delimiters
Separate content

Refusal
Handle unsafe asks

STRUCTURED OUTPUT

PR

Schema
Constrain fields

Enum
Limit choices

Required
Prevent omissions

Type
Protect parsing

Stop sequence
End generation

Parser
Validate output

Repair
Retry invalid JSON

TOOL SAFETY

TO

Name, schema, scope, auth, errors.

Name | Schema | Scope | Auth | Errors

TOOL VS RESOURCE

TO

Tool	Resource
Performs action	Provides context
Needs arguments	Read oriented

Action needs tool

MCP PRIMITIVES

TO

Tool
Callable action

Resource
Readable context

Prompt
Reusable template

Server
Expose capabilities

stdio
Local subprocess

HTTP
Remote service

Auth
Protect access

STDIO VS HTTP

TO

stdio	HTTP
Local process	Remote service
Developer workflow	Network access

Match deployment boundary

PRODUCTION READY

RE

Eval, retry, timeout, monitor, fallback.

Eval | Retry | Timeout | Monitor | Fallback

EVAL VS MONITOR

RE

Eval	Monitor
Predeployment test	Production signal
Known cases	Live outcomes

Use both

RELIABILITY PICKER

RE

Output must parse →	Schema validation
Facts may change →	Retrieve current source
Latency matters →	Smaller model
Cost matters →	Cache prefix
Tool may fail →	Structured error
Safety matters →	Guardrails
Behavior drifts →	Regression evals
Ambiguity remains →	Ask user

CONTEXT + RETRIEVAL

RE

Context window
Token budget

Compression
Keep essentials

Retrieval
Fetch relevant facts

Chunking
Split documents

Citation
Expose provenance

Caching
Reuse static prefix

Freshness
Check current facts

PRODUCTION RELIABILITY

RE

Eval
Measure behavior

Golden set
Known examples

Regression
Catch drift

Retry
Handle transient failure

Timeout
Bound latency

Fallback
Degrade gracefully

Monitor
Track outcomes

COMMON TRAPS

Overagenting
Simple call works ≠ Avoid needless loops

Broad Tools
Scope permissions ≠ Constrain inputs

Unparsed Output
Validate schemas ≠ Repair failures

Stale Context
Retrieve current facts ≠ Cite sources

No Evaluation
Define success ≠ Track regressions

LAST MINUTE

1. Start with baseline
2. Pick pattern deliberately
3. Scope tool access
4. Validate structured output
5. Handle tool errors
6. Budget context tokens
7. Add eval coverage
8. Plan fallback behavior

Independent exam preparation notice

OpenExamPrep is an independent education provider, not affiliated with or endorsed by the exam sponsors named on this site. Exam and certification names identify what our resources help you study for; they do not imply sponsorship, approval, or review of our materials. Unless a resource expressly identifies authorized public material, our practice questions are independently developed simulations, not actual, recalled, or sponsor-provided exam items. We aim to reflect publicly available exam outlines and topic information in our study resources. Coverage, format, and difficulty may differ from the actual exam, and we cannot guarantee that every detail is accurate or current. Confirm exam requirements, fees, and policies with the official exam sponsor.

Updated 2026-10-20. Official exam sources checked Exam guide v1.0, effective July 2026. This is a study aid only and is not for use during the exam.